Languages and Algorithms for AI
Modulo 2 – Logica, LP e CLP

Giovanni "Qua' Qua' dancer" Palma

Contents

Chapter 1 **Introduzione: AI simbolica e bisogno della logica** Page

1.1 Organizzazione del corso
Programma del modulo — • Testi —

1.2 Due esempi motivazionali
Esempio 1: un problema combinatorio — • Esempio 2: un problema di ottimizzazione (Cakes) —

1.3 Programmazione dichiarativa

1.4 Che cos'è la logica
Tante logiche diverse —

1.5 Perché studiare logica in informatica
1) L'informatica deriva dalla logica — • 2) Fondamento della computazione e dei linguaggi dichiarativi — • 3) Fondamento dell'informatica teorica (calcolabilità) — • 4) Dimostrare la correttezza del software — • 5) La logica come linguaggio privo di paradossi —

1.6 Perché studiare logica in AI

1.7 Che cos'è l'AI: un po' di storia
Le quattro visioni dell'AI (Russell–Norvig) —

1.8 AI simbolica vs. sub-simbolica
AI simbolica: i sistemi esperti — • Logica e AI — • AI sub-simbolica —

Chapter 2 **La nozione di verità e di conseguenza logica** Page

2.1 Che cos'è la verità
La verità in matematica — • Conseguenze — • Riassunto —

2.2 Conseguenza ed equivalenza logica
Conclusioni sulla verità —

2.3 Logica proposizionale
I connettivi —

2.4 Approfondimento: sintassi e semantica (Van Dalen §1.1–1.2)
Sintassi — • Semantica — • Legami tra $\models$, connettivi ed equivalenza — • Alcune equivalenze notevoli —

Chapter 1

Introduzione: AI simbolica e bisogno della logica

Modulo 2 – M. Gabbrielli, DISI, Università di Bologna. Lucidi: `lucidi/1.Intro.Logica`. Materiale di approfondimento: Van Dalen, *Logic and Structure*; Bratko, *Prolog Programming for AI*; Gabbrielli–Martini, capitolo su LP e CLP; Tutorial MiniZinc; Russell & Norvig, cap. 1.

1.1 Organizzazione del corso

Note:

- 1° modulo: Felli (sulla slide il prof ha corretto a mano "Lodi" → "Felli").
- 2° modulo: questo (Gabbrielli).
- Il corso integrato con Dal Lago è stato **cancellato** a mano dal prof: non fa parte di questo esame.
- Esami **separati** per ogni modulo; per questo modulo l'esame è **solo scritto**.
- Parte del materiale viene dal corso di logica (in italiano) di **Claudio Sacerdoti Coen** (UniBo), nome scritto a mano sulla copertina; il resto da Russell–Norvig, Van Dalen, E. Clarke.

1.1.1 Programma del modulo

1. AI simbolica e bisogno della logica.
2. Introduzione alla logica: logica proposizionale, logica del primo ordine (FOL), risoluzione, unificazione.
3. Logic Programming: interpretazione procedurale, interpretazione dichiarativa, Prolog (prima parte; la seconda in un altro corso).
4. Constraint Programming: CLP, MiniZinc (prima parte).

1.1.2 Testi

- Dispense (su Virtuale).
- D. van Dalen, *Logic and Structure*, 4ª ed., Springer — riferimento per la parte di logica.
- I. Bratko, *Prolog Programming for Artificial Intelligence*.
- M. Triska, *The Power of Prolog* (online).
- Russell, Norvig, *Artificial Intelligence: A Modern Approach*.
- *The MiniZinc Handbook* (online).

1.2 Due esempi motivazionali

Prima di parlare di logica il prof mostra due problemi che si risolvono *descrivendoli* invece di scrivere un algoritmo:

1. un problema **combinatorio** (risolto in Prolog);
2. un problema di **ottimizzazione** (risolto in MiniZinc).

1.2.1 Esempio 1: un problema combinatorio

Definition 1.2.1: Il problema (sequenza di Langford a tre copie)

Disporre in una sequenza tre 1, tre 2, ..., tre 9 (in totale 27 numeri) in modo che, per ogni $i \in [1,9]$, tra due occorrenze *successive* di i ci siano **esattamente** i **numeri**.

Example 1.2.1 (Una soluzione)

$$1,9,1,2,1,8,2,4,6,2,7,9,4,5,8,6,3,4,7,5,3,9,6,8,3,5,7$$

Verifica per $i=1$: gli 1 stanno nelle posizioni $1,3,5$; tra due 1 consecutivi c'è esattamente un numero. Per $i=2$: posizioni $4,7,10$, quindi due numeri in mezzo. (Sulla slide il prof sottolinea a colori le occorrenze di ciascun numero.)

Il programma Prolog che lo risolve:

```
sequence([_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_]).
% sequence(X) -> X e' una lista di 27 variabili

question(S) :-
    sequence(S),
    sublist([9,_,_,_,_,_,_,_,_,9,_,_,_,_,_,_,_,_,9], S),
    sublist([8,_,_,_,_,_,_,_,8,_,_,_,_,_,_,_,8], S),
    sublist([7,_,_,_,_,_,_,7,_,_,_,_,_,_,7], S),
    sublist([6,_,_,_,_,_,6,_,_,_,_,_,6], S),
    sublist([5,_,_,_,_,5,_,_,_,_,5], S),
    sublist([4,_,_,_,4,_,_,_,4], S),
    sublist([3,_,_,3,_,_,3], S),
    sublist([2,_,2,_,2], S),
    sublist([1,_,1,_,1], S).
% S e' una soluzione
```

Note:

Come leggerlo (approfondimento, Bratko cap. 1–3).

- `_` è una **variabile anonima**: ogni occorrenza è una variabile nuova, diversa dalle altre.
- Il programma *non dice come* cercare la soluzione: dice solo *che cosa* è una soluzione. S è una lista di 27 elementi, e per ogni i il "pattern" $[i, \underbrace{_,\dots,_}_{i}, i, \underbrace{_,\dots,_}_{i}, i]$ deve comparire come **sottolista contigua** di S.
- `sublist/2` non è un predicato standard di SWI-Prolog; una definizione possibile è

  ```
  sublist(L, S) :- append(_, T, S), append(L, _, T).
  ```

  cioè L è sottolista di S se $S = A \cdot L \cdot B$ per qualche A, B.
- Il "come" lo fa l'interprete: **unificazione** (il pattern viene "incollato" su un tratto di S, legando le variabili) e **backtracking** (se un `sublist` successivo fallisce si torna indietro e si prova un'altra posizione). Mettere prima il 9 (il pattern più lungo) riduce le posizioni possibili e quindi lo spazio di ricerca.

1.2.2 Esempio 2: un problema di ottimizzazione (Cakes)

Definition 1.2.2: Il problema delle torte

Si possono fare due tipi di torte:

1. **banana**: 250 g farina, 2 banane, 75 g zucchero, 100 g burro;
2. **cioccolato**: 200 g farina, 75 g cacao, 150 g zucchero, 150 g burro.

Prezzi di vendita: cioccolato \$4.50, banana \$4.00. Disponibilità: 4 kg farina, 6 banane, 2 kg zucchero, 500 g burro, 500 g cacao. **Domanda**: quante torte di ciascun tipo fare per **massimizzare il profitto**?

```
% Baking cakes for the school fete
var 0..100: b;   % no. of banana cakes
var 0..100: c;   % no. of chocolate cakes

constraint 250*b + 200*c <= 4000;   % flour
constraint 2*b <= 6;                % bananas
constraint 75*b + 150*c <= 2000;    % sugar
constraint 100*b + 150*c <= 500;    % butter
constraint 75*c <= 500;             % cocoa

solve maximize 400*b + 450*c;       % profit (in cents)

output ["no. of banana cakes = \(b)\n",
        "no. of chocolate cakes = \(c)\n"];
```

Note:

Struttura di un modello MiniZinc (Tutorial MiniZinc).

- `var 0..100: b;` dichiara una **variabile decisionale** intera con dominio $[0, 100]$;
- `constraint` aggiunge un **vincolo** che ogni soluzione deve rispettare;
- `solve maximize` f chiede una soluzione **ottima** rispetto a f (con `solve satisfy` se ne chiede una qualsiasi);
- i prezzi sono scritti in centesimi per restare negli interi.

Anche qui scriviamo solo *che cosa* vogliamo; il solver decide come esplorare le combinazioni.

Example 1.2.2 (La soluzione ottima (approfondimento))

Il vincolo stringente è il burro: $100b + 150c \leq 500$, cioè $2b + 3c \leq 10$; inoltre le banane danno $b \leq 3$. Le coppie ammissibili "sul bordo" sono:

(b, c)	burro	$400b + 450c$
$(3, 1)$	450	1650
$(2, 2)$	500	**1700**
$(1, 2)$	400	1300
$(0, 3)$	450	1350

Gli altri vincoli (farina, zucchero, cacao) sono soddisfatti da tutte. L'ottimo è $b = 2$, $c = 2$, profitto \$17.00.

1.3 Programmazione dichiarativa

Definition 1.3.1: Algorithm = Logic + Control (Kowalski)

Un algoritmo si scompone in due parti:

- **Logic**: *che cosa* va calcolato (la specifica del problema);
- **Control**: *come* lo si calcola (ordine dei passi, strategia di ricerca).

Definition 1.3.2: Programmazione tradizionale vs. dichiarativa

- Nella programmazione **tradizionale** (imperativa) il programmatore si occupa sia della Logic sia del Control.
- Nella programmazione **dichiarativa** il programmatore si occupa **solo della Logic**; il Control lo gestisce l'interprete del linguaggio.

Il compito del programmatore è *dichiarare* il problema: **cosa** vogliamo ottenere e non **come** ottenerlo.

Definition 1.3.3: Tre paradigmi dichiarativi

1. **Logic Programming** (Prolog, ...): basato sulla nozione di *relazione*.
2. **Functional programming** (ML, Haskell, OCaml, ...): basato sulla composizione di *funzioni*.
3. **Constraint Programming** (MiniZinc, CLP, ILOG, ...): basato su relazioni e su *solver* specifici per un dominio.

Note:

"Constraint programming represents one of the closest approaches computer science has yet made to the Holy Grail of programming: the user states the problem, the computer solves it." — E. C. Freuder.
Approfondimento (Gabbrielli–Martini). In Prolog il Control è fissato: le clausole si provano dall'alto in basso, i letterali del corpo da sinistra a destra, con ricerca in profondità e backtracking. Per questo in pratica l'ordine delle clausole *conta* (può cambiare efficienza e perfino terminazione), e la separazione Logic/Control in Prolog non è perfetta. Il significato dichiarativo (l'insieme delle risposte *vere*) resta però indipendente da quest'ordine.

1.4 Che cos'è la logica

Definition 1.4.1: Logica

La logica è lo studio sistematico della **forma delle inferenze valide** e delle leggi più generali della **verità**. Un'**inferenza valida** è una in cui esiste una specifica relazione di supporto logico tra le *premesse* e la *conclusione*.

Tradizionalmente la logica comprende: classificazione degli argomenti, esposizione della "forma logica" comune a tutti gli argomenti validi, studio della dimostrazione e dell'inferenza (compresi paradossi e fallacie), studio di **sintassi** e **semantica**. È studiata in filosofia (dall'antichità), in matematica (da metà Ottocento) e oggi nelle scienze cognitive (informatica, linguistica, filosofia, psicologia).

1.4.1 Tante logiche diverse

Ogni logica fissa *che cosa* conserva un'inferenza. Con premesse X, Y e conclusione W:

Logica	Domanda	Uso in CS/AI
Classica (prop., FOL)	se X e Y sono veri, W è vero?	AI, linguaggi logici, verifica
Intuizionista	da prove di X, Y costruisco una prova di W?	software corretto per costruzione
Lineare	avendo X e Y posso *scambiarli* per W?	controllo delle risorse
Epistemica	se credo che tu sappia X, Y, credo W?	sicurezza
Temporale	se X, Y accadranno, W potrà accadere?	specifica e verifica

Note:

Logiche diverse, ma problemi e tecniche simili. La logica è interdisciplinare (filosofia, matematica, informatica, AI). **Logica matematica** = applicazione del metodo matematico allo studio della logica, *ma anche* logica come fondamento della matematica (logiche diverse $\Rightarrow$ matematiche diverse).

1.5 Perché studiare logica in informatica

Cinque motivi (numerati a mano dal prof sulla slide):

1. l'informatica deriva dalla logica;
2. la logica fonda la computazione e i linguaggi dichiarativi;
3. la logica fonda l'informatica teorica (calcolabilità);
4. serve a dimostrare la correttezza del software;
5. la logica è un linguaggio privo di paradossi.

1.5.1 1) L'informatica deriva dalla logica

Question 1: È nato prima il computer o l'informatica?

La risposta sembra ovvia, ma l'informatica esiste **prima e indipendentemente** dal computer, come **studio della calcolabilità**.

La storia, in breve:

- La matematica antica (geometria a parte) non aveva basi formali rigorose; queste vengono cercate nella logica e nell'aritmetica.
- Si applicano metodi matematici allo studio della logica: la derivazione logica viene identificata con un processo di calcolo (*calculemus!*).
- Studiando i processi di calcolo si scoprono risultati sorprendenti: esistono funzioni **non calcolabili** (anzi, sono la maggioranza), ed esiste una classe di funzioni calcolabili definibile in modi del tutto diversi (lambda calcolo, macchine di Turing, programmi logici, RAM).
- Intorno al 1930 nasce l'informatica.

Note:

"Computer Science is no more about computers than Astronomy is about telescopes." — E. W. Dijkstra.
Il fatto che formalismi così diversi definiscano *la stessa* classe di funzioni è alla base della **tesi di Church–Turing**: le funzioni "calcolabili da un procedimento effettivo" sono esattamente quelle calcolabili da una macchina di Turing.

1.5.2 2) Fondamento della computazione e dei linguaggi dichiarativi

Due grandi classi di linguaggi: **dichiarativi** (funzionali, logici, a vincoli) e **imperativi**.

- **Linguaggi funzionali** (LISP, ML, Haskell, OCaml): basati sulla composizione di funzioni. Legame forte con la logica: *programmare corrisponde a dimostrare* (in una logica opportuna) e *i tipi corrispondono ad enunciati di teoremi* (corrispondenza di Curry–Howard).
- **Linguaggi logici** (Prolog, CLP, CHR, CCP): basati sulla nozione di relazione. Legame ancora più forte: **computazione = deduzione**; Algorithm = Logic + Control.
- **Linguaggi a vincoli** (CLP, CHR, CCP, MiniZinc): relazioni + solver specifici di dominio; esistono sia basati sulla logica sia imperativi; molto usati in applicazioni pratiche.

1.5.3 3) Fondamento dell'informatica teorica (calcolabilità)

Due domande:

1. Data un'affermazione vera o falsa, possiamo sempre *dimostrarla*?
2. Data una funzione generica, possiamo *calcolarla* con un computer?

La risposta è **no** in entrambi i casi: la logica studia i limiti della conoscenza. Non tutto si può dimostrare vero o falso (incompletezza), non tutto si può calcolare. Sono limiti *inevitabili* per l'informatica e l'AI.

1.5.4 4) Dimostrare la correttezza del software

Question 2: Possiamo essere sicuri che un generico programma scritto in un linguaggio comune sia corretto?

Theorem 1.5.1 Risultato fondamentale

La maggior parte delle proprietà (non banali) dei programmi **non è decidibile**. In particolare non può esistere uno strumento automatico che, dato un programma generico (in un linguaggio Turing-completo), decida se è corretto (ad esempio se termina).

Note:

Questo è il **teorema di Rice** (approfondimento): ogni proprietà non banale della *funzione calcolata* da un programma è indecidibile. Il caso della terminazione è dimostrato più sotto (Teorema 1.5.2).

Come se ne esce:

- si scrive una **specifica** e si usa la logica per dimostrare che il programma la soddisfa;
- **prove automatiche**: solo in casi specifici o su *approssimazioni* delle proprietà;
- **prove assistite**: l'umano fa la dimostrazione (in logica), il computer la *verifica*.

Note:

Note a mano del prof: il disegno della "trusted code base".

- Dato un programma (il foglio) e un oracolo "?" che dovrebbe dire se è corretto: un tale oracolo è **non decidibile** (croce rossa).
- Strada alternativa: l'umano scrive la *specifica* e la *dimostrazione* in logica (il riquadro con $\forall, \exists, \Rightarrow$); un piccolo programma **verificatore di prove** controlla la dimostrazione e risponde sì/no (spunta verde).
- Controllare una prova data è *decidibile*, trovarla in generale no. Ci si deve fidare solo del verificatore, che è piccolo e semplice: questa è la **trusted code base** (evidenziata in giallo). È l'idea dei proof assistant (Coq/Rocq, Lean, Isabelle).

Definition 1.5.1: Specifica di un algoritmo di ordinamento (dagli appunti del prof)

Pre-condizione: l'input è un array b di interi.
Post-condizione: l'output è un array a di interi tale che

$$(\forall i < a.length - 1.\ a[i] \le a[i+1]) \ \wedge\ (\forall i < a.length.\ \exists j.\ a[i] = b[j]) \ \wedge\ (\forall i < b.length.\ \exists j.\ b[i] = a[j]).$$

La prima congiunzione dice che a è **ordinato**; le altre due che a e b contengono **gli stessi valori**.

Note:

Osservazione (approfondimento). Le ultime due clausole dicono "stessi valori" ma *non* "stesse molteplicità": l'output $a = [1,2,2]$ soddisfa la specifica con input $b = [2,1,1]$. La specifica corretta richiede che a sia una **permutazione** di b (stessa lunghezza e, per ogni valore x, lo stesso numero di occorrenze). Esempio di quanto è facile sbagliare una specifica: anche la specifica va verificata. (Negli appunti gli indici sono scritti con $\le$; con array indicizzati da 0 servono le disuguaglianze strette, come sopra.)

Tre limiti delle funzioni calcolate da un computer

Nota importante: i risultati seguenti valgono per linguaggi **Turing-completi**, cioè con il massimo potere espressivo noto.

Proposition 1.5.1 Limite 1: le funzioni calcolate non sono solo totali

Definiamo nel linguaggio

$$f(g) = \texttt{not}\,(g(g))$$

e applichiamo f a sé stessa: $f(f) = \texttt{not}\,(f(f))$. Se $f(f)$ restituisse un booleano v avremmo $v = \neg v$, assurdo. Quindi la computazione di $f(f)$ **non termina**: le funzioni calcolate dai programmi sono in generale **parziali**.

Note:

Accanto a questa slide il prof ha scritto

$$X = \{Y \mid Y \notin Y\} \qquad X \in X \iff X \notin X$$

cioè il **paradosso di Russell** (Sezione 1.5.5): $f(g) = \texttt{not}(g(g))$ è la stessa costruzione, con "g applicata a g" al posto di "$Y \in Y$". In matematica genera un paradosso; in un linguaggio di programmazione genera una funzione *parziale*.

Theorem 1.5.2 Limite 2: la terminazione non è decidibile

Non esiste un programma f (in alcun linguaggio) tale che, per ogni programma g e argomento x,

$$f(g,x) = \begin{cases} \texttt{true} & \text{se } g(x) \text{ termina,} \\ \texttt{false} & \text{se } g(x) \text{ non termina.} \end{cases}$$

Dimostrazione: Per assurdo, supponiamo che f esista. Definiamo

```
h(g) = if f(g,g) = true then while true do skip else return 0
```

e consideriamo $h(h)$:

$$h(h) \text{ termina} \iff f(h,h) = \texttt{false} \iff h(h) \text{ non termina.}$$

Contraddizione, quindi f non può esistere. ☺

Note:

f deve essere essa stessa *totale* (risponde sempre) perché l'argomento funzioni: è proprio questo che la rende impossibile. La struttura è di nuovo la **diagonalizzazione**: h applicata a sé stessa fa il contrario di quello

che f prevede.

Lenma 1.5.1 I programmi sono numerabili

L'insieme dei programmi esprimibili in un qualsiasi linguaggio di programmazione è **numerabile**.

Dimostrazione: Un programma è una stringa finita su un alfabeto finito Σ. Le stringhe di lunghezza n sono $|\Sigma|^n$, un numero finito. Elencando prima le stringhe di lunghezza 0, poi quelle di lunghezza 1, poi 2, ... (ciascun gruppo in ordine lessicografico) si ottiene un'enumerazione di Σ^*. I programmi sono un sottoinsieme di Σ^*, quindi sono al più numerabili.

Theorem 1.5.3 Le funzioni booleane non sono numerabili

Sia $F = \{f \mid f : \mathbb{N} \to \{0,1\}\}$. Allora F non è numerabile.

Dimostrazione (diagonale): Per assurdo, sia $f_1, f_2, f_3, \ldots$ un'enumerazione di tutte le funzioni di F. Definiamo

$$f^*(n) = \begin{cases} 1 & \text{se } f_n(n) = 0, \\ 0 & \text{se } f_n(n) = 1. \end{cases}$$

Allora $f^* \in F$, ma per ogni i si ha $f^*(i) \neq f_i(i)$, quindi $f^* \neq f_i$ per ogni i. Dunque f^* non compare nell'enumerazione, contro l'ipotesi che fosse completa.

Corollary 1.5.1 Limite 3: i linguaggi esprimono pochissime funzioni

Nessun linguaggio di programmazione può esprimere tutte le funzioni booleane $\mathbb{N} \to \{0,1\}$ (e quindi, a fortiori, tutte le funzioni matematiche).

Dimostrazione: I programmi sono numerabili (lemma), F no (teorema): non può esistere una suriezione dai programmi su F.

Dimostrazione alternativa: il diagonale di Cantor

Theorem 1.5.4 Cantor

Sia T un insieme con almeno due elementi. Allora $|T| < |T^T|$, dove T^T è l'insieme delle funzioni $T \to T$.

Dimostrazione: Non può essere $|T| > |T^T|$: la mappa $t \mapsto$ (funzione costante t) è iniettiva da T in T^T. Basta dunque mostrare $|T| \neq |T^T|$.
Per assurdo, sia $g : T \to T^T$ una biiezione. Definiamo $f \in T^T$ ponendo, per ogni $t \in T$, $f(t) = t'$ con $t' \neq g(t)(t)$ (esiste perché T ha almeno due elementi). Allora per ogni t si ha $f(t) \neq g(t)(t)$, quindi $f \neq g(t)$. Dunque f non è nell'immagine di g, che non è suriettiva: assurdo.

Corollary 1.5.2 Fatto 3, versione alternativa

Sia T l'insieme di tutti i valori di un linguaggio, che contiene almeno i booleani e le funzioni. Se le funzioni del linguaggio fossero *tutte* le funzioni matematiche $T \to T$, avremmo $T \supseteq \{0,1\} \cup T^T$, quindi $|T| \geq |T^T|$, contro Cantor. Quindi un linguaggio non può esprimere tutte le funzioni matematiche.

1.5.5 5) La logica come linguaggio privo di paradossi

Definition 1.5.2: Paradosso

Un **paradosso** è un ragionamento apparentemente corretto (rispetto a regole e assiomi dati), da premesse apparentemente corrette, che porta a una conclusione apparentemente contraddittoria.

Il linguaggio naturale permette di esprimere paradossi, quindi non è adatto come base per la matematica o l'informatica. La logica fornisce linguaggi privi di paradossi.

Example 1.5.1 (Falso paradosso (trova l'errore))

$$x = 1 \Rightarrow x^2 = x \Rightarrow x^2 - 1 = x - 1 \Rightarrow (x-1)(x+1) = x - 1 \Rightarrow x + 1 = 1 \Rightarrow x = 0.$$

L'errore: si divide per $x - 1$, che vale 0.

Example 1.5.2 (Il mentitore)

"Io sono un bugiardo." Sono bugiardo sse quello che dico non è vero, sse "io sono un bugiardo" non è vero.

Example 1.5.3 (Paradosso di Berry)

"Sia x il più piccolo numero che non può essere definito con meno di 1000 parole." Allora x è definito (con meno di 1000 parole, quelle della frase) sse x non è definibile con meno di 1000 parole.

Definition 1.5.3: Metalinguaggio

Un **metalinguaggio** è un linguaggio usato per parlare di un linguaggio. "Metalinguistico": relativo a un metalinguaggio.

Proposition 1.5.2 Perché nascono questi paradossi

1. uso **metalinguistico** del linguaggio naturale (la frase parla di frasi, di definizioni, ...);
2. **auto-applicazione** di un concetto metalinguistico (la frase parla di sé stessa).

Rimedio: abbandonare il linguaggio naturale e usare un linguaggio opportuno della logica matematica, in cui linguaggio oggetto e metalinguaggio sono separati.

Paradossi in matematica

Theorem 1.5.5 Paradosso di Russell

Sia $X = \{Y \mid Y \notin Y\}$. Allora $X \in X \iff X \notin X$.

Note:

Soluzione: $\{Y \mid Y \notin Y\}$ *non è un insieme.*

- Non si può usare la nozione di insieme in modo metalinguistico: la collezione di tutti gli insiemi non è un insieme.
- Gli insiemi si costruiscono solo **selezionando elementi da un insieme già dato** (assioma di separazione di ZF: $\{Y \in A \mid \varphi(Y)\}$). Così $\{Y \in$ classe di tutti gli insiemi $\mid Y \notin Y\}$ è una **classe**, non un insieme, e quindi non contiene sé stessa.

Paradossi in informatica

Molti linguaggi ammettono funzioni di ordine superiore (che prendono funzioni in input): sono caratteristiche metalinguistiche che rendono possibile l'auto-applicazione. Qui i paradossi **non sono evitabili**; ma invece di produrre contraddizioni mostrano i *limiti* dei nostri modelli di calcolo (funzioni parziali, indecidibilità: vedi i Limiti 1–3).

Il paradosso di Curry

Example 1.5.4 (Versione in linguaggio naturale)

A = "Se questa frase è vera, allora Babbo Natale esiste"; sia B = "Babbo Natale esiste". Per dimostrare A (un'implicazione) assumiamo la premessa, "questa frase è vera", cioè A. Ma A è $A \to B$, quindi per modus ponens B. Abbiamo dimostrato $A \to B$, cioè A. Ora A è vera e $A \to B$ è vera: per modus ponens B. Babbo Natale esiste.

Proposition 1.5.3 Curry in un contesto logico

1. $X := (X \to Y)$	assunzione ("se questa frase è vera allora Y")
2. $X \to X$	legge di identità
3. $X \to (X \to Y)$	da 2, sostituendo il lato destro con 1
4. $X \to Y$	da 3 per *contrazione*
5. X	da 4, per la definizione 1
6. Y	da 5 e 4 per modus ponens

Note:

È di nuovo un paradosso? Dipende dal passo 1. In molte logiche, se Y è falso, la 1 non è possibile. In logica proposizionale classica (approfondimento): se Y è falso, $X \leftrightarrow (X \to Y)$ diventa $X \leftrightarrow \neg X$, che non è soddisfatta da nessuna valutazione. Quindi non esiste alcuna formula X con $X \equiv (X \to Y)$ e Y falso: la "definizione" 1 non è esprimibile, e la derivazione è corretta ma parte da un'ipotesi impossibile. Il problema nasce solo se il linguaggio permette a una frase di riferirsi a sé stessa.
(Sulla slide la derivazione è barrata a mano dal prof.)

1.6 Perché studiare logica in AI

a) È uno strumento per **esprimere il ragionamento umano**.

b) È il fondamento di un **approccio classico all'AI** (vedi anche il corso Foundations).

c) È il fondamento dei **linguaggi (dichiarativi) usati in AI**.

d) Ha applicazione diretta in linguaggi usati in AI, come **Prolog e CLP** (questo corso).

1.7 Che cos'è l'AI: un po' di storia

Come l'informatica, anche l'AI esiste *prima* dei computer.

- **Ramon Llull** (1232–1316), *Ars combinatoria*: dischi di carta che combinano concetti rappresentati simbolicamente.
- **Cartesio** (1596–1650): influenzato dagli automi esposti a Parigi, studia il rapporto mente–corpo (**dualismo**: il corpo funziona come una macchina, la mente non è materiale, interagiscono nella ghiandola pineale).

Precursore delle scienze cognitive e dell'AI perché considera la costruzione di automi un processo cognitivo importante.

- **Leibniz** (1646–1716), *calculus ratiocinator*: un linguaggio universale in cui risolvere ogni controversia calcolando (*"calculemus!"*).
- XVIII secolo: automi di **Vaucanson** (l'anatra che "mangia") e **Jaquet-Droz** (lo scrivano e il musicista, con movimenti controllati da un *disco metallico*: un programma ante litteram).
- Il **Turco meccanico** (XVIII sec.): un "automa" che giocava a scacchi, in realtà con un giocatore umano nascosto. *AI ante litteram... fake!* (Il nome è stato ripreso da Amazon Mechanical Turk, lavoro umano nascosto dietro servizi "automatici".)
- **Turing**: 1935 macchina universale; 1947 talk "What we want is a machine that can learn from experience"; 1948 *Intelligent Machinery* (mai pubblicato in vita, un vero manifesto dell'AI con sistemi simbolici e reti neurali); 1950 *Computing Machinery and Intelligence* (imitation game).
- **Neuroni e apprendimento**: McCulloch–Pitts (1943), primo modello computazionale del neurone (logica a soglia); Hebb (1949), apprendimento hebbiano; Rosenblatt (1958), **perceptron**.
- **1956, Dartmouth**: John McCarthy conia il termine *Artificial Intelligence*: "ogni aspetto dell'apprendimento o dell'intelligenza può in principio essere descritto così precisamente da poter costruire una macchina che lo simuli". McCarthy inventa anche il LISP.

1.7.1 Le quattro visioni dell'AI (Russell–Norvig)

Thinking humanly	**Thinking rationally**
Acting humanly	**Acting rationally**

Definition 1.7.1: Thinking humanly

Sistemi che **pensano come gli umani**: imitare il cervello. Non ne sappiamo abbastanza (Broca 1861, Golgi 1873, EEG 1929, fMRI 1990): conosciamo relazioni tra aree e funzioni, non come funzionano un processo cognitivo o la memoria. L'approccio delle **scienze cognitive** (anni '60, "rivoluzione cognitiva" contro il comportamentismo) richiede teorie validate o top-down (predire e testare il comportamento di soggetti umani) o bottom-up (dati neurologici). Oggi è distinto dall'AI.

Definition 1.7.2: Thinking rationally

Sistemi che **pensano razionalmente**: le "leggi del pensiero", cioè la **logica**. Approccio *normativo* (prescrittivo) e non descrittivo, da Aristotele in poi. Problemi: (1) non tutto il comportamento intelligente passa per la deliberazione logica; (2) qual è lo scopo del pensare, quali pensieri *dovrei* avere?

Definition 1.7.3: Acting rationally

Sistemi che **agiscono razionalmente**: fare la cosa giusta, cioè quella che ci si aspetta massimizzi il raggiungimento dell'obiettivo date le informazioni disponibili. Non richiede necessariamente il pensiero (es. il riflesso di chiudere gli occhi), ma il pensiero dovrebbe essere al servizio dell'azione razionale.

Definition 1.7.4: Acting humanly: il test di Turing

Turing (1950) sostituisce "le macchine possono pensare?" con "le macchine possono *comportarsi* in modo intelligente?", e propone un test operativo: l'**imitation game**. Un interrogatore dialoga per scritto con un umano e una macchina e deve capire chi è chi.

Note:

Turing prevedeva che entro il 2000 una macchina avrebbe avuto il 30% di probabilità di ingannare una persona comune per 5 minuti; anticipò le principali obiezioni all'AI e indicò le componenti chiave: conoscenza, ragionamento, linguaggio, apprendimento. **Problema**: il test non è riproducibile, costruttivo, né analizzabile matematicamente.
Sul "solipsismo" Turing osserva che l'unico modo di sapere se una macchina pensa sarebbe *essere* la macchina; ma lo stesso varrebbe per gli altri esseri umani. Invece di discutere all'infinito, "è usuale avere la cortese convenzione che tutti pensino".

Note:

Posizione del corso: da informatici adottiamo il punto di vista "semplice" di Turing, e consideriamo AI i sistemi che agiscono umanamente (o almeno razionalmente).

Definition 1.7.5: Definizione ufficiale di AI

L'AI è la capacità di un computer di svolgere compiti comunemente associati a esseri intelligenti. Due approcci principali:

1. **AI simbolica**;
2. **AI sub-simbolica** (connessionista, incluso il ML).

Note:

Peter Norvig: l'AI è *"knowing what to do when you don't know what to do"*.

1.8 AI simbolica vs. sub-simbolica

1.8.1 AI simbolica: i sistemi esperti

Example 1.8.1 (Un sistema per le infezioni batteriche (stile MYCIN))

Sys: la coltura è Gram positiva? **User**: NO.
Sys: è Gram negativa? **User**: SÌ.
Sys: il paziente è allergico agli antibiotici? **User**: NO.
Sys: suggerirei una terapia a base di antibiotici.

Non è ChatGPT: il sistema **spiega** le proprie decisioni. Alla domanda **WHY?** (perché ti serve questa informazione?) risponde indicando la regola che sta cercando di applicare; alla domanda **HOW?** (come hai dedotto questo?) risponde indicando la regola usata e le premesse.

Definition 1.8.1: Sistema esperto

Un **sistema esperto** (anni '70 in poi):

- usa conoscenza di dominio fornita da umani, tipicamente come **regole logiche** (**knowledge base**);
- risolve problemi in un dominio limitato con prestazioni simili a un esperto;
- cerca e costruisce dinamicamente la soluzione con un **motore inferenziale** che elabora le regole.

Proposition 1.8.1 Caratteristiche della computazione simbolica

I. ragionamento basato sulla logica;
II. può fornire **spiegazioni** delle decisioni;
III. non adatta a problemi come il riconoscimento di immagini (che non si risolvono algoritmicamente).

Formalizzare la diagnosi in logica

Obiettivo: suggerire un farmaco adatto al paziente, cioè dimostrare $prescribe(Drug)$.
Fatti: $gram(neg)$, $\neg allergic(antb)$.
Regole:

R1: $gram(neg) \rightarrow id(ecoli)$ (Gram-negativo $\Rightarrow$ enterium-coli)

R2: $gram(pos) \rightarrow id(strep)$ (Gram-positivo $\Rightarrow$ streptococco)

R3: $id(strep) \vee id(bact) \rightarrow ind(pen)$ (strepto o batterio $\Rightarrow$ indicata la penicillina)

R4: $ind(X) \wedge \neg allergic(X) \rightarrow prescribe(X)$

R5: $id(ecoli) \rightarrow ind(antb)$

Definition 1.8.2: Motore inferenziale (backward chaining)

Si parte dal **goal** e si va all'indietro:

1. il goal $prescribe(Drug)$ fa *match* con la testa di R4 ($Drug/X$); si attiva R4: nuovi sotto-goal $ind(X)$ e $\neg allergic(X)$;
2. $ind(X)$ fa match con R3 (X/pen) e con R5 ($X/antb$): due rami alternativi;
3. ramo R3: serve $id(strep)$ (via R2, serve $gram(pos)$: **NO**) oppure $id(bact)$ (nessuna regola: **NO**). Il ramo fallisce;
4. ramo R5: serve $id(ecoli)$, via R1 serve $gram(neg)$: è un fatto, **SÌ**; con $X = antb$ anche $\neg allergic(antb)$ è un fatto: **SÌ**.

Risposta: $prescribe(antb)$, con il legame $X/antb$ (annotato a mano dal prof sull'albero).

Note:

Approfondimento: lo stesso in Prolog. È esattamente il modo di ragionare di Prolog (risoluzione SLD: goal-directed, depth-first, sinistra-destra, backtracking):

```
gram(neg).
not_allergic(antb).

id(ecoli)  :- gram(neg).          % R1
id(strep)  :- gram(pos).          % R2
ind(pen)   :- id(strep).          % R3 (due clausole per l'OR)
ind(pen)   :- id(bact).
ind(antb)  :- id(ecoli).          % R5
prescribe(X) :- ind(X), not_allergic(X).   % R4

?- prescribe(Drug).
Drug = antb.
```

Osservazioni: la disgiunzione nel corpo di R3 diventa *due clausole*; il fatto negativo $\neg allergic$ è stato reso con un predicato positivo, perché le clausole di Prolog (clausole di Horn) non ammettono negazioni nei fatti. Le spiegazioni WHY/HOW corrispondono a mostrare il ramo dell'albero di derivazione.

1.8.2 Logica e AI

- Molte logiche: proposizionale, primo ordine, modali (epistemica, temporale, deontica), ...
- Molte tecniche di ragionamento: **deduzione**, **induzione**, **abduzione**.
- Molti linguaggi e strumenti: Prolog, constraint programming.
- Base di molti sistemi esperti.

Note:

Deduzione, induzione, abduzione (approfondimento). Con regola $A \to B$:

- *deduzione*: da A e $A \to B$ concludo B (corretta);
- *abduzione*: da B e $A \to B$ ipotizzo A come spiegazione (es. diagnosi: dal sintomo alla causa; non corretta in generale);
- *induzione*: da molti casi di A e B insieme generalizzo $A \to B$ (è ciò che fa il ML).

1.8.3 AI sub-simbolica

Proposition 1.8.2 Caratteristiche della computazione sub-simbolica

I. basata su reti neurali (artificiali) e ML;

II. molto buona per riconoscimento di immagini e problemi non risolvibili algoritmicamente;

III. **non può fornire spiegazioni** delle decisioni (e quindi è ancora lontana dal superare il test di Turing).

Example 1.8.2 (Riconoscimento di tumori in istologia)

Train: da immagini di vetrini interi si estraggono patch etichettate (tumore / normale) e si addestra un modello profondo. **Test**: il vetrino nuovo viene diviso in patch sovrapposte, il modello restituisce una probabilità per patch, e si ottiene una *mappa di probabilità* del tumore. Il modello indica *dove*, ma non *perché*.

Chapter 2

La nozione di verità e di conseguenza logica

Lucidi: `lucidi/2.Truth`. *Approfondimento: Van Dalen,* Logic and Structure, *§1.1 (Propositions and Connectives) e §1.2 (Semantics).*

2.1 Che cos'è la verità

Abbiamo visto che la logica è lo studio sistematico della forma delle **inferenze valide** e delle leggi più generali della **verità**. Ma che cos'è la verità?

Definition 2.1.1: Verità (senso comune)

La verità è l'essere **in accordo con i fatti o con la realtà** (oppure la fedeltà a un originale o a uno standard).

Nelle scienze sperimentali (fisica, chimica, biologia) la verità è associata al **mondo sensibile**:

- un **esperimento ripetibile** definisce una verità;
- se il mondo sensibile cambia, cambia anche la verità.

Question 3: Che cosa succede quando non c'è un mondo sensibile, ad esempio in matematica?

2.1.1 La verità in matematica

Le nostre "verità probabili" sono, ad esempio:

- la somma degli angoli interni di un triangolo è 180°;
- in modo equivalente, il **V postulato** di Euclide: dati una retta e un punto fuori di essa, esiste **una e una sola** retta per il punto parallela alla retta data.

Però...

Example 2.1.1 (Geometria sferica (Riemann, 1854))

Reinterpretiamo i termini primitivi della geometria:

- **punto** = punto sulla superficie di una sfera;
- **retta** = cerchio massimo, cioè intersezione della sfera con un piano che passa per il centro.

Allora:

- dati due punti esiste una sola retta che li contiene;
- due rette hanno sempre almeno due punti in comune (quindi *non esistono parallele*);
- la somma degli angoli interni di un triangolo è **maggiore di** 180°.

Note:

Precisazione (approfondimento). "Dati due punti esiste una sola retta" vale per punti *non antipodali*: per due punti opposti (es. i poli) passano infiniti cerchi massimi. Per questo nella *geometria ellittica* si identificano i punti antipodali. Il triangolo con un vertice al polo nord e due sull'equatore, distanti un quarto di giro, ha tre angoli retti: somma 270°.

2.1.2 Conseguenze

Proposition 2.1.1 Il metodo assiomatico descrive modelli, non il mondo

- Il metodo assiomatico **non è una descrizione del mondo sensibile**.
- La geometria (la matematica) non sembra fornire giudizi sintetici a priori (nel senso di Kant).
- Fornisce invece la descrizione di un **insieme di mondi (modelli)** in cui:
  1. si possono interpretare le entità (punti, rette, ...) come si vuole, *purché gli assiomi siano rispettati*;
  2. mondi diversi hanno proprietà diverse;
  3. solo alcune di queste proprietà sono dimostrabili dagli assiomi (**incompletezza**).

Note:

Il punto 3 si legge così: il V postulato è vero nel piano euclideo e falso sulla sfera, e *entrambi* soddisfano gli altri assiomi. Quindi il V postulato **non è dimostrabile** dagli altri assiomi (altrimenti sarebbe vero in tutti i loro modelli), e neanche la sua negazione. È un enunciato *indipendente*. Questo è esattamente il metodo che useremo in logica: per mostrare che F *non* segue da Γ basta esibire *un* mondo in cui Γ è vero e F è falso (un **controesempio**).

Per parlarne con precisione introduciamo un po' di lessico.

Definition 2.1.2: Teoria, modello, consistenza

- Una **teoria** è un insieme di enunciati (assiomi).
- Un **modello** di una teoria è un mondo, con un'interpretazione dei simboli, in cui tutti gli assiomi sono veri.
- Una teoria è **consistente** se da essa non si può derivare l'assurdo.

Proposition 2.1.2 Impatto sulla visione della matematica

- Una teoria non si accetta o scarta perché è soddisfatta (banalmente) dal mondo sensibile, ma perché **ammette dei modelli**.
- Una teoria che ammette modelli **non è mai inconsistente**: da essa non si può derivare l'assurdo.
- La nozione assoluta di verità (data dal mondo sensibile) viene sostituita dalla nozione di **conseguenza logica**.

Note:

Perché "ha un modello ⇒ consistente"? Le regole di deduzione sono **corrette** (*soundness*): trasformano enunciati veri in un mondo in enunciati veri nello stesso mondo. In un modello tutti gli assiomi sono veri, quindi è vero tutto ciò che si deriva; ma l'assurdo $\bot$ non è vero in nessun mondo, quindi non si deriva. Il viceversa (consistente ⇒ ha un modello) è il **teorema di completezza**, che vedremo (Van Dalen §1.5).

2.1.3 Riassunto

- **Verità del mondo sensibile**: cambia quando cambia il mondo (mutamenti *ristretti*: il mondo cambia secondo le sue leggi).
- **Verità della matematica**: cambia adottando nuovi mondi (mutamenti *non ristretti*: qualsiasi mondo che rispetta gli assiomi).
- Inoltre la verità cambia se si **interpretano** in modo diverso le parole di un enunciato.

Question 4: Che cosa resta della nozione di verità se il mondo cambia arbitrariamente?

Resta ciò che *non dipende* dal mondo: la conseguenza logica.

2.2 Conseguenza ed equivalenza logica

Definition 2.2.1: Conseguenza logica (lucidi)

Sia $\Gamma = \{F_1, \ldots, F_n\}$ un insieme di enunciati (**ipotesi** o **premesse**) e F un enunciato (**conclusione**). F è **conseguenza logica** di Γ, e si scrive

$$\Gamma \models F,$$

se, per tutte le interpretazioni delle formule nei vari mondi possibili, vale sempre: *se tutte le formule di* Γ *sono vere (nel mondo in esame) allora anche* F *è vera.*

Note:

Come leggerla.

- Non si chiede che F sia vera: si chiede che sia vera *in ogni mondo in cui le premesse sono vere.*
- I mondi in cui qualche premessa è falsa non contano.
- $\Gamma \not\models F$ significa che esiste *almeno un* mondo (un controesempio) in cui tutte le premesse sono vere e F è falsa.
- Con $\Gamma = \emptyset$: $\models F$ significa che F è vera in tutti i mondi (F è **valida**, o **tautologia** nel caso proposizionale).

Definition 2.2.2: Equivalenza logica

F è **logicamente equivalente** a G, e si scrive $F \equiv G$, sse F è conseguenza logica di G e G è conseguenza logica di F:

$$F \equiv G \iff F \models G \text{ e } G \models F.$$

Lenma 2.2.1 Caratterizzazione dell'equivalenza

$F \equiv G$ sse in ogni mondo (con ogni interpretazione) F e G hanno lo **stesso valore di verità**.

Dimostrazione: ($\Rightarrow$) Fissato un mondo: se F è vera, da $F \models G$ anche G è vera; se G è vera, da $G \models F$ anche F è vera. Quindi F è vera sse G è vera: stesso valore.
($\Leftarrow$) Se in ogni mondo F e G hanno lo stesso valore, in ogni mondo in cui F è vera lo è anche G ($F \models G$) e viceversa ($G \models F$). ☺

Theorem 2.2.1 L'equivalenza logica è una relazione di equivalenza

(Esercizio dei lucidi.) Per tutte le formule F, G, H:

1. **riflessività**: $F \equiv F$;
2. **simmetria**: $F \equiv G \Rightarrow G \equiv F$;
3. **transitività**: $F \equiv G$ e $G \equiv H \Rightarrow F \equiv H$.

Dimostrazione: Basta mostrare che $\models$ tra singole formule è riflessiva e transitiva; la simmetria di $\equiv$ viene gratis dalla definizione.

1. $F \models F$: in ogni mondo in cui F è vera, F è vera. Quindi $F \models F$ e $F \models F$, cioè $F \equiv F$.
2. $F \equiv G$ significa "$F \models G$ e $G \models F$", che è la stessa cosa di "$G \models F$ e $F \models G$", cioè $G \equiv F$.
3. Transitività di $\models$: se $F \models G$ e $G \models H$, prendiamo un mondo in cui F è vera; allora G è vera (da $F \models G$), quindi H è vera (da $G \models H$). Dunque $F \models H$. Allo stesso modo da $H \models G$ e $G \models F$ si ha $H \models F$. Quindi $F \equiv H$.

(In alternativa: per il lemma, $F \equiv G$ sse "stesso valore in ogni mondo", e "avere lo stesso valore" è chiaramente riflessiva, simmetrica e transitiva.)

2.2.1 Conclusioni sulla verità

- La verità è sempre definita **rispetto a un mondo** e a una data **interpretazione** dei simboli in quel mondo.
- La verità di un singolo enunciato **non è interessante**: cambia quando cambia il mondo.
- Ciò che interessa sono la **conseguenza logica** e l'**equivalenza logica**: **non cambiano** al variare del mondo.

2.3 Logica proposizionale

La logica proposizionale è la logica più semplice; illustra le idee di base usando le **proposizioni**.

Definition 2.3.1: Proposizione atomica e valore di verità

Una **proposizione** è un enunciato *dichiarativo*, a cui si può (in linea di principio) assegnare un valore vero o falso. Esempi:

- P_1: la neve è bianca;
- P_2: oggi piove;
- P_3: questo corso è noioso.

Ogni P_i che non ha parti che siano a loro volta proposizioni è un **atomo** (o formula atomica). Ogni P_i è **vero o falso, ma mai entrambi** (logica a due valori). Il valore vero o falso assegnato a una proposizione è il suo **valore di verità**.

Note:

(Van Dalen §1.1) Non serve sapere *di fatto* quale sia il valore (la congettura di Goldbach è una proposizione anche se non sappiamo se è vera); basta che un valore ci sia. Domande e ordini ("chiudi la porta!") non sono proposizioni: non hanno valore di verità.

2.3.1 I connettivi

Il linguaggio naturale ha molti connettivi: *and, or, not, but, since, if... then*, ... Perché la logica ne usa pochi?

Example 2.3.1 ("But" vs. "and")

- "Max è marxista, *ma* non è privo di umorismo."
- "π è irrazionale, *ma* non è algebrico."

Nel primo "ma" porta un significato emotivo (sorpresa). Nel secondo la sorpresa è meno immediata (bisognerebbe sapere che "quasi tutti" gli irrazionali sono algebrici) e si può usare "e" senza cambiare significato.

Per **economia** scegliamo solo connettivi utili per esprimere le dimostrazioni, come "e" e "o". Ma anche questi sono ambigui.

Example 2.3.2 (Problema 1: "and" e il tempo)

- John è ripartito *e* ha investito un pedone.
- John ha investito un pedone *e* è ripartito.

In italiano "e" può avere una funzione di **ordinamento temporale** (le due frasi dicono cose diverse). In matematica e in logica no: $A \wedge B$ significa solo che valgono entrambe, e $A \wedge B$ e $B \wedge A$ dicono la stessa cosa.

Example 2.3.3 (Problema 2: "if... then" e "or")

1. Se apro la finestra allora avremo aria fresca.
2. Se apro la finestra allora $1 + 3 = 4$.
3. Se $1 + 2 = 4$ allora avremo aria fresca.
4. John sta lavorando o è a casa.
5. Euclide era un greco o un matematico.

- In 1 c'è una **relazione** tra premessa e conclusione.
- In 2 e 3 non c'è alcuna relazione, eppure in matematica sono perfettamente accettabili: è l'**implicazione materiale**.
- Tendiamo ad accettare 4 e a rifiutare 5: nel parlato "o" suona esclusivo, e non si usa "o" quando si potrebbe dire "e". In logica "o" è **inclusivo**: 5 è vera.

Proposition 2.3.1 Conclusione

Serve un **linguaggio artificiale** con un significato **preciso** dei connettivi.

Note:

Perché l'implicazione materiale (Van Dalen §1.2). "Se Smith è dentro allora Jones è dentro" è sicuramente *falsa* solo quando Smith è dentro e Jones è fuori. Quando Smith è fuori, l'informazione non è falsa, è solo "poco utile"; avendo deciso che ogni enunciato è vero o falso, la si dichiara vera. È una *scelta*, motivata dall'eleganza del sistema che ne risulta. Da ricordare: **c'è un solo caso in cui un'implicazione è falsa** (premessa vera, conclusione falsa). Quindi 2 è vera (conclusione vera) e 3 è vera (premessa falsa).

2.4 Approfondimento: sintassi e semantica (Van Dalen §1.1–1.2)

Formalizziamo ora quanto detto. Si procede in due passi: (1) si definisce il **linguaggio** (sintassi); (2) si definisce quando una formula è **vera** (semantica). La nozione informale di "mondo con un'interpretazione" diventa, nel caso proposizionale, una **valutazione**.

2.4.1 Sintassi

Definition 2.4.1: Alfabeto della logica proposizionale

(i) simboli proposizionali: $p_0, p_1, p_2, \ldots$;

(ii) connettivi: $\wedge$ (and), $\vee$ (or), $\to$ (implicazione), $\neg$ (not), $\leftrightarrow$ (sse), $\bot$ (falsum, l'assurdo);

(iii) simboli ausiliari: ( ).

I simboli proposizionali e $\bot$ sono gli **atomi**.

Definition 2.4.2: Proposizioni (formule)

L'insieme $PROP$ è il **più piccolo** insieme X tale che:

(i) $p_i \in X$ per ogni i, e $\bot \in X$;

(ii) se $\varphi, \psi \in X$ allora $(\varphi \wedge \psi), (\varphi \vee \psi), (\varphi \to \psi), (\varphi \leftrightarrow \psi) \in X$;

(iii) se $\varphi \in X$ allora $(\neg\varphi) \in X$.

Note:

"Il più piccolo" significa: in $PROP$ ci sono *solo* le stringhe ottenibili applicando un numero finito di volte le regole (i)–(iii). Le lettere greche φ, ψ *non* sono formule: sono **metavariabili**, cioè simboli del metalinguaggio che stanno per formule qualsiasi. È la separazione linguaggio/metalinguaggio che evita i paradossi del capitolo precedente.

Theorem 2.4.1 Principio di induzione sulle formule

Sia A una proprietà. Se

(i) $A(\varphi)$ vale per ogni atomo φ,

(ii) $A(\varphi)$ e $A(\psi)$ implicano $A(\varphi \square \psi)$ per $\square \in \{\wedge, \vee, \to, \leftrightarrow\}$,

(iii) $A(\varphi)$ implica $A(\neg\varphi)$,

allora $A(\varphi)$ vale per ogni $\varphi \in PROP$.

Dimostrazione: Sia $X = \{\varphi \in PROP \mid A(\varphi)\}$. Per (i)–(iii), X soddisfa le tre clausole della definizione di $PROP$. Poiché $PROP$ è il più piccolo insieme con queste proprietà, $PROP \subseteq X$, cioè A vale per tutte le formule. ☺

2.4.2 Semantica

Usiamo 1 per "vero" e 0 per "falso".

Definition 2.4.3: Valutazione

Una funzione $v : PROP \to \{0,1\}$ è una **valutazione** se

$$\begin{aligned} v(\varphi \wedge \psi) &= \min(v(\varphi), v(\psi)), \\ v(\varphi \vee \psi) &= \max(v(\varphi), v(\psi)), \\ v(\varphi \to \psi) = 0 &\iff v(\varphi) = 1 \text{ e } v(\psi) = 0, \\ v(\varphi \leftrightarrow \psi) = 1 &\iff v(\varphi) = v(\psi), \\ v(\neg\varphi) &= 1 - v(\varphi), \\ v(\bot) &= 0. \end{aligned}$$

Le stesse regole come **tavole di verità**:

φ	ψ	$\neg\varphi$	$\varphi \wedge \psi$	$\varphi \vee \psi$	$\varphi \to \psi$	$\varphi \leftrightarrow \psi$
0	0	1	0	0	1	1
0	1	1	0	1	1	0
1	0	0	0	1	**0**	0
1	1	0	1	1	1	1

Theorem 2.4.2 Estensione unica

Se v è una funzione dagli atomi in $\{0,1\}$ con $v(\bot) = 0$, esiste un'**unica** valutazione $[\![\cdot]\!]_v$ tale che $[\![\varphi]\!]_v = v(\varphi)$ per ogni atomo φ.

Note:

Si definisce $[\![\cdot]\!]_v$ per ricorsione sulla struttura della formula, usando le clausole della definizione di valutazione; l'unicità si dimostra per induzione. Conseguenza pratica: **una valutazione è determinata dai valori sugli atomi**. Nel caso proposizionale un "mondo" è quindi semplicemente un'assegnazione di 0/1 alle proposizioni atomiche.

Lenma 2.4.1 Dipendenza dai soli atomi presenti

Se $v(p_i) = v'(p_i)$ per tutti i p_i che compaiono in φ, allora $[\![\varphi]\!]_v = [\![\varphi]\!]_{v'}$.

Dimostrazione: Induzione su φ. Se φ è un atomo p_i è l'ipotesi; se è $\bot$ entrambe valgono 0. Se $\varphi = \varphi_1 \square \varphi_2$, gli atomi di φ_1 e φ_2 sono atomi di φ, quindi per ipotesi induttiva $[\![\varphi_j]\!]_v = [\![\varphi_j]\!]_{v'}$ per $j = 1, 2$, e il valore di φ si calcola dagli stessi valori con la stessa regola. Analogo per $\neg$.

Corollary 2.4.1 Decidibilità

Se φ contiene n atomi distinti, per sapere se è vera in tutte le valutazioni basta controllarne 2^n (le righe della tavola di verità). Quindi la validità proposizionale è **decidibile** (ma con costo esponenziale nel caso peggiore).

Definition 2.4.4: Tautologia e conseguenza semantica

(i) φ è una **tautologia**, e si scrive $\models \varphi$, se $[\![\varphi]\!]_v = 1$ per ogni valutazione v.

(ii) Sia Γ un insieme di formule. $\Gamma \models \varphi$ sse per ogni v: se $[\![\psi]\!]_v = 1$ per ogni $\psi \in \Gamma$, allora $[\![\varphi]\!]_v = 1$. Si dice che φ è **conseguenza semantica** di Γ.

Convenzione: $\varphi_1, \ldots, \varphi_n \models \psi$ sta per $\{\varphi_1, \ldots, \varphi_n\} \models \psi$.

Note:

È esattamente la definizione dei lucidi, con "mondo + interpretazione" sostituito da "valutazione".

Example 2.4.1 (Tautologie e conseguenze (Van Dalen))

- $\models \varphi \to \varphi, \quad \models \neg\neg\varphi \to \varphi, \quad \models \varphi \vee \psi \leftrightarrow \psi \vee \varphi$;
- $\varphi, \psi \models \varphi \wedge \psi$;
- $\varphi, \varphi \to \psi \models \psi$ (*modus ponens*);
- $\varphi \to \psi, \neg\psi \models \neg\varphi$ (*modus tollens*).

Verifica di modus ponens: le valutazioni con $[\![\varphi]\!] = 1$ e $[\![\varphi \to \psi]\!] = 1$ sono solo quelle con $[\![\psi]\!] = 1$ (riga 1, 1 della tavola).

Example 2.4.2 (Un controesempio: affermazione del conseguente)

$\varphi \to \psi, \psi \not\models \varphi$: con $v(\varphi) = 0$ e $v(\psi) = 1$ le premesse sono vere e la conclusione è falsa. (È l'inferenza dell'*abduzione*: plausibile ma non corretta.)

2.4.3 Legami tra $\models$, connettivi ed equivalenza

Proposition 2.4.1 Equivalenza e bi-implicazione

$\varphi \equiv \psi$ sse $\models \varphi \leftrightarrow \psi$.

Dimostrazione: Per il lemma di caratterizzazione, $\varphi \equiv \psi$ sse $[\![\varphi]\!]_v = [\![\psi]\!]_v$ per ogni v, sse $[\![\varphi \leftrightarrow \psi]\!]_v = 1$ per ogni v (definizione di $\leftrightarrow$), sse $\models \varphi \leftrightarrow \psi$. ☺

Theorem 2.4.3 Teorema di deduzione semantico

$\Gamma, \varphi \models \psi$ sse $\Gamma \models \varphi \to \psi$.

Dimostrazione: ($\Rightarrow$) Sia v che rende vere tutte le formule di Γ. Se $[\![\varphi]\!]_v = 0$, allora $[\![\varphi \to \psi]\!]_v = 1$. Se $[\![\varphi]\!]_v = 1$, v rende vero $\Gamma \cup \{\varphi\}$, quindi $[\![\psi]\!]_v = 1$ e $[\![\varphi \to \psi]\!]_v = 1$.
($\Leftarrow$) Sia v che rende vero $\Gamma \cup \{\varphi\}$. Allora $[\![\varphi \to \psi]\!]_v = 1$ e $[\![\varphi]\!]_v = 1$, quindi (unico caso falso dell'implicazione escluso) $[\![\psi]\!]_v = 1$. ☺

Corollary 2.4.2 Conseguenza e validità

$\varphi_1, \ldots, \varphi_n \models \psi$ sse $\models (\varphi_1 \wedge \cdots \wedge \varphi_n) \to \psi$.

Theorem 2.4.4 Conseguenza e insoddisfacibilità

$\Gamma \models \varphi$ sse $\Gamma \cup \{\neg\varphi\}$ non ha alcuna valutazione che rende vere tutte le sue formule (è **insoddisfacibile**).

Dimostrazione: $\Gamma \not\models \varphi$ sse esiste v con Γ tutto vero e $[\![\varphi]\!]_v = 0$, sse esiste v con Γ tutto vero e $[\![\neg\varphi]\!]_v = 1$, sse $\Gamma \cup \{\neg\varphi\}$ è soddisfacibile. Negando entrambi i lati si ottiene la tesi. ☺

Note:

Quest'ultimo teorema è l'idea su cui si basano la **risoluzione** e quindi Prolog: per dimostrare che un goal segue dal programma, si aggiunge la sua *negazione* e si cerca di derivare la contraddizione (la clausola vuota). Lo vedremo nelle prossime lezioni.

2.4.4 Alcune equivalenze notevoli

Theorem 2.4.5 Leggi algebriche (Van Dalen 1.3.1, 1.3.4)

Sono tautologie:

- associatività e commutatività di $\wedge$ e $\vee$;
- distributività: $\varphi \vee (\psi \wedge \sigma) \leftrightarrow (\varphi \vee \psi) \wedge (\varphi \vee \sigma)$ e $\varphi \wedge (\psi \vee \sigma) \leftrightarrow (\varphi \wedge \psi) \vee (\varphi \wedge \sigma)$;
- De Morgan: $\neg(\varphi \vee \psi) \leftrightarrow \neg\varphi \wedge \neg\psi$ e $\neg(\varphi \wedge \psi) \leftrightarrow \neg\varphi \vee \neg\psi$;
- doppia negazione: $\neg\neg\varphi \leftrightarrow \varphi$;
- $(\varphi \to \psi) \leftrightarrow (\neg\varphi \vee \psi)$;
- $(\varphi \leftrightarrow \psi) \leftrightarrow (\varphi \to \psi) \wedge (\psi \to \varphi)$;
- $\neg\varphi \leftrightarrow (\varphi \to \bot)$.

Dimostrazione (De Morgan, le altre sono analoghe): $[\![\neg(\varphi \vee \psi)]\!] = 1 \iff [\![\varphi \vee \psi]\!] = 0 \iff [\![\varphi]\!] = [\![\psi]\!] = 0 \iff [\![\neg\varphi]\!] = [\![\neg\psi]\!] = 1 \iff [\![\neg\varphi \wedge \neg\psi]\!] = 1$, per ogni valutazione. ☺

Example 2.4.3 (Esercizio svolto: contrapposizione)

$(\varphi \to \psi) \equiv (\neg\psi \to \neg\varphi)$. Infatti

$$\neg\psi \to \neg\varphi \equiv \neg\neg\psi \vee \neg\varphi \equiv \psi \vee \neg\varphi \equiv \neg\varphi \vee \psi \equiv \varphi \to \psi.$$

Ogni passo usa una delle leggi sopra; il passaggio da un'equivalenza all'altra è lecito perché $\equiv$ è transitiva (teorema precedente) e perché sostituire una sottoformula con una equivalente dà una formula equivalente (*teorema di sostituzione*, Van Dalen 1.2.6).

Example 2.4.4 (Esercizio svolto: Curry rivisto)

Esiste una valutazione con $[\![X \leftrightarrow (X \to Y)]\!] = 1$ e $[\![Y]\!] = 0$? Con $[\![Y]\!] = 0$ si ha $[\![X \to Y]\!] = 1 - [\![X]\!]$, quindi servirebbe $[\![X]\!] = 1 - [\![X]\!]$: impossibile. Dunque $X \leftrightarrow (X \to Y) \models Y$: l'"assunzione 1" del paradosso di Curry implica già Y da sola, ed è soddisfacibile solo nei mondi dove Y è vero. Nessun paradosso.